

$$= E[(f - E(f))^2] + \text{var}(y) + \text{var}(\hat{f})$$

$$= \text{Bias}(\hat{f})^2 + \text{var}(y) + \text{var}(\hat{f})$$

$$\boxed{\text{MSE} = \text{Bias}[\hat{f}]^2 + \sigma^2 + \text{var}(\hat{f})}$$

- A variety of techniques to avoid overfitting:
 - Cross validation
 - Regularization
 - Feature selection
 - Dimensionality reduction

Cross-validation:

- In conventional validation
 - Data partitioned into training, validation and test data set.
 - validation set is used to experiment and tune parameters of learnt model or to compare performances of prediction algorithms (created using the training set) and choose the algo. with best performance.
- Validation can be enough if, even after partition enough training samples are present and need rough performance estimate.
 - otherwise cross-validation preferable.
- If we have small data set, we will sequence the data set.
- Cross validation is a model evaluation method that checks how well a model generalizes to new data.

- In one round of cross-validation
 - Data divided into 2 subsets
 - One for training & one for testing
 - Testing performance is recorded.
- Multiple rounds of cross-validation are performed under diff. partitions.
 - Testing results from all rounds are averaged to generate a more accurate estimate of model prediction performance.
- Takes $n\%$ of data as test data and tests the model, it trains with remaining data, put back that data and take $n\%$ again diff data & check.
- Cross-validation helps to identify if model behaves poorly due to problems like over-fitting.
- Model should ignore noise, if not able to do then it is over-fitting model.

we did not use all partitions.
Non-exhaustive Cross-validation:

- 2 popular cross-validation schemes
 - Non-exhaustive & exhaustive.
 - more popular
- k -fold cross-validation quite popular
 - original data split into k equal-sized folds.
 - In each trial, one fold becomes testing set, rest becoming training set
 - Repeat the process k -times

• The final avg. or
Holdout

- split set
- Issue: testing multiple
- Testing average performance
- Exhaustive

- Leave k in k
 - let
 - For n
 - slow
 - Better
 - ev
 - ea
 - ev

- The final data to check can be the avg. or the total data set.

Holdout method:

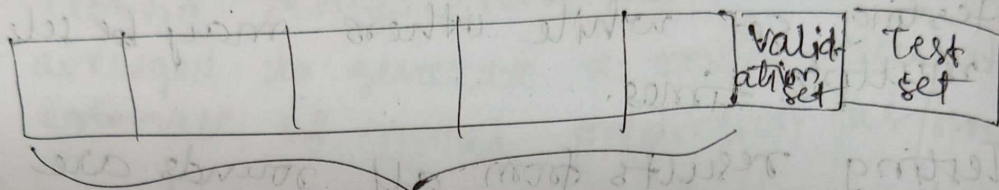
- split data into training and testing set numerous times
- Issue: Some samples may never end up in testing set while others may be selected multiple times.
- Testing results from all rounds are averaged to generate model prediction performance.

Exhaustive Cross-validation:

- leave out one cross " (LOOCV) [when k in k -fold set to # of data points].
→ want to identify all partitions
- let each data be in the testing set once.
- For dataset of size n , LOOCV needs n rounds of cross-validation [can be slow when n is large].
- Benefit of so many fit and evaluated models is a more robust estimate of model performance as each row of data can represent the entirety of the test dataset.

Nested Cross-validation:

- Popular way to tune parameters of an algo.
- One version: k -fold cross validation with validation and test set.
- Lets say parameter X needs tuning.



Train set

- $k=7$ in our example
 - one set each picked as Test & validation ($k=2$), picked for training.
 - for the picked Test set
 - perform k -fold cross-validation on T & V set here ($k=6$)
 - compute the avg. training error for each value of X
 - pick the best X .
- $k=7$
 - Get the set with least errors.
 - need to find the best X ?
 - $k=7$, 1-test, 1-val
 - Removal
 - $k=6$, ~~data~~ perform cross-val. for each X value.
- Repeat for each possible Test set [i.e., 7 times]

- Pick X times
 - no. of
- Regular
- Unbiased source
- Regular the error to be
- Principles 1
 - On sim model no
- Less Comp
- However that points as its that
- Comp the data

• Pick X that was returning maximum times to order loop.

→ No. of cross validating ? $\rightarrow 5 \times 6 \times 7 = 1$

$$\frac{1}{2} 11 \propto 11^2$$

Regularization

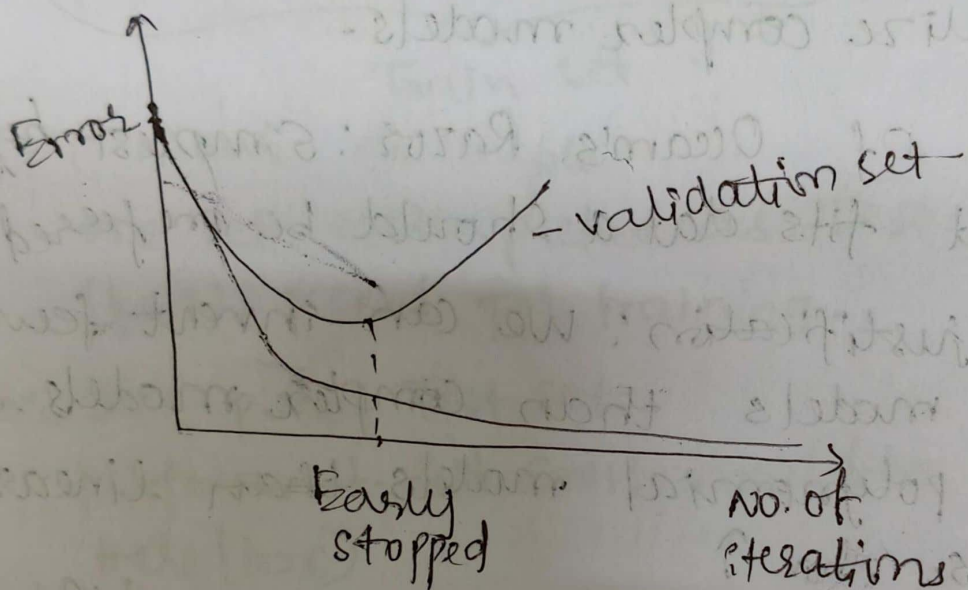
- Unnecessary complexity of model is a source of overfitting.
- Regularization adds extra parameters to the error f^n we are trying to minimize to penalize complex models.
- Principle of Occam's Razor: Simplest hypothesis that fits data should be preferred
 - One justification: We can invent fewer simple models than complex models. e.g., more polynomial models than linear models. $\uparrow$
- Less solⁿs in simple spaces (Nature!) than Complex space.
- However, it is much easier to find a model that perfectly captures all the training data points with a higher order polynomial f^n as its search space is much larger than that of linear model.
- Complex models generate worse since they were fine-tuned to fit specific data points, hence more prone to overfitting.

- Simpler models need less computation time.
- Linear model is preferable since it may generalize better.

- Stopping training procedure early can also be a form of regularization.

— Since it is likely to produce simpler model.

— Approach called Early stopping.



— During starting phase, the error is high, it learns and that rate decreases. But also learns unnecessary things like all sentences. When we test it may not work properly.

- Regularization needs to be kept on a moderate level.
- When too small doesn't make any impact & it too big, results in underfitting as it moves model from ground truth.

• Let $\hat{f}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$

• we want to minimize the MSE:

$$\frac{1}{m} \times \min_{\theta_0, \theta_1, \theta_2, \theta_3} \sum_{i=1}^m (\hat{f}_\theta(x^{(i)}) - y^{(i)})^2$$

• where m is the no. of training samples,
 θ 's are the weight parameters.

• Let MSE be represented by $I(\theta)$.

• Let's say we want to penalize the higher order (2 and 3).

• Can add penalty terms say $+1000\theta_2 + 1000\theta_3$

• The effect of this would be that θ_2 and θ_3 need to be quite small to minimize error.

• A significantly high penalty can actually convert a overfit problem to an underfit problem.

— Since all the terms with high regularization parameter would become 0 or close to 0.

• E.g.: If all terms except θ_0 have a high enough regularization parameter then $\hat{f}(x)$, can become a constant!!!

≡

Feature selection

- Feature selection is the process of picking a subset of significant features for use in better model construction.
- Not all features are useful and can add randomness to our results.
- Feature selection in principle boils down to whether to include a feature or not.
- For n features we have 2^n feature sets.
- Identifying meaningful features.
- Either start with all features and remove features iteratively or start with min. set of features & add features iteratively.

Filter methods: features are selected on the basis of their scores in various statistical tests for their correlation and with the outcome variable.

E.g: Chi squared test (for categorical variables) or Pearson correlation coefficient scores (for continuous variables).

- Data collⁿ, Data preprocessing, Data cleaning takes most of the time.
- Data testing
- Interpretability & Explainability.

↓
You want ~~atg~~ ML model to tell the granular parts like parameters used for an also used

- EX: If 2 variables are highly correlated, then no need of both, remove.
- If 2 variables are highly correlated and if feature is highly correlated to outcome, don't remove.
- If any variable is less correlated to outcome, remove the feature.

Wrapper methods: Try to use a subset of features and train a model using them. Based on the inferences from the previous model, decide to add or remove features from the subset. The problem is essentially reduced to a search algorithm.

Recursive feature elimination:

- If eliminate worst, we get the order of worst features.
- "select best" "best features."
- Top 5/7 features, 90-95% able to explain.
- selects a subset by performing and the algo. and either eliminates worst or best performing feature at each iteration. It will construct the next model with the left features until all the features are exhausted.
- It ranks the features based on the order of their stimulation.

Embedded methods:

- Learn which features best contribute to the accuracy of the model while the model is being created.
- Not only before model starts running, but also during the running phase.

Dimensionality Reduction: — Minimize losses.

- Transform high dim^l data into lower dim^l space.
- Need dim^l reduction. — loss of information, lossy compression.
- Ex: Need 2D from 3D, have to chuck in diff. dirⁿs
- Transformation will lead to info. loss.

Data Preprocessing:

- Cross Industry Standard Process of data mining (CRISPDM). — Popular methodology

- CRISP DM: ^{client should} ^{satisfy} actual high level problem in data mining.

• Business understanding: Business person formulates a business problem (domain experts).

• Data understanding: A technical analyst also gets involved along with domain experts. Usually called exploration phase involves understanding complicated data.

ML focusses on modeling.
Data mining focusses on data topic. Most steps comes related to data.

- Data Preparation

- Modeling

- Evaluation

- Deployment

Feature Engineering: - can be

- Features can be of many types, can follow distributions e.g: normal, binomial
- Can be binary y/N, +/- etc.-
- Can be categorical - high/med/low, set of continents
- can be quantitative e.g: temp in °C, price in \$.
- Feature engg. is process of creating or improving features.

12/2/25

Data Processing

Data Preparation: Phase where we create training and test datasets. Can call "preprocessing phase". - Data cleaning also.

Modeling: formulate a model to fit and is usually associated with ML.

↳ finding an algorithm suitable.

Evaluation: Evaluate the model and data to check if the business problem is solved.

↳ Both parties involved.

Deployment: Setting up the system in a production environment.

Key to all this is high quality data:

→ cleaning data is therefore important

→ cleaning data can form large part of ML many times.

Data preprocessing:

- No one way to clean data, involves manual work.
- Visualize the data.
- Data can be grid of numbers, images, audio, text or something else.
- Grid of numbers is most convenient form and hence ideal to work to have numerical features.
 - Table of no's is assumed default unless specified otherwise.

Feature Engineering: (Already wrote in previous page).

- ↳ Improve features, removed unnecessary features.
- Features often based on common sense domain knowledge or prior experience.
 - More art than science.
- Missing values: Data can have missing

values for some features -
- sometimes easy to find by just scanning
counting etc.

↳ Data can have gaps in real-life data.

• How to deal with missing values?

- Ignore missing values.

- Substitute missing values by fixed values, called "imputing" e.g., impute the arithmetic mean, median or mode.

• Humans deal with various types of values, ML algo's in general deal with numerical values.

• Label Encoding: Refers to converting labels into numeric form to make it ML readable.

- Eg., Height can have Tall, Medium and Short as categories.

- Using label encoding can convert to 0, 1 and 2.

↳ Giving labels to data

• Process can be called integer-label encoding

- Issue: Learner may conclude there is an ordering.

• One-hot-encoding or one-of-k: Refers to splitting the column which contains numerical categorical data to many columns depending on the no. of categories present in that column.

- Each column contains "0" or "1" corresponding to which column it is being placed.

• Scaling: Values of features can differ by orders of magnitude. May need to scale many times. Some common strategies:

- Standardization: Remove mean of feature and divide by standard deviation. If feature values normally distributed, we get Gaussian centered around 0 with variance of 1.

- If feature values are not normally distributed, remove median and divide by the interquartile range i.e., range b/w the first (25th) and third (75th) quartile.

- Scaling features to range is commonly used - common range used is 0 to 1.

• Polynomial features: Given two features a and b , polynomial relation of form

$$a^2 + ab + b^2$$

↳ interaction form.

- Occam's razor: Avoid higher order poly and interaction of many features.

- complex polys are hard to compute

- little additional benefit

• Can decompose the model prediction as:

- Constant term (1,50,000); (from example)
- An effect for the size feature (+100,000 if big; 40 if small)
- An effect for the location (+50,000 if good; 0 if bad)
- This decomposition fully explains the model predictions.

→ Approximation.

• Power transformations: Functions to transform numerical features into a more convenient form.

e.g.: Conform better or closer to normal distribution.

- Box-Cox transform tries to find the best power needed to transform into data that is closer to normal distribution.

→ Can lead to loss of info., losing original data

• Binning: Separate feature values into bins

e.g.: Precipitation values can be converted into rain or no-rain or high/medium/low/no bins.

- Binning can lead to loss of information but improve on speed and memory requirements.

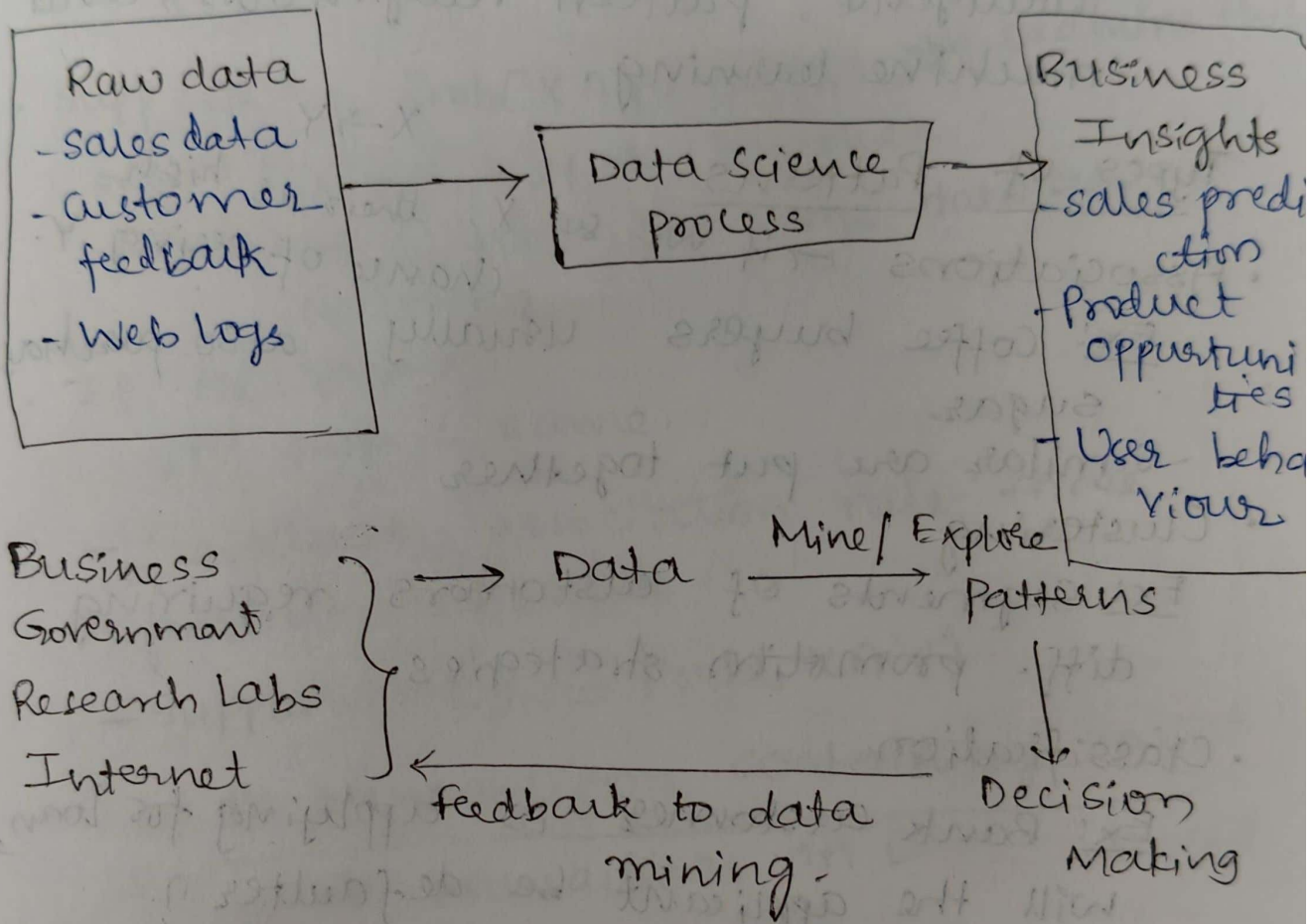
- Can reduce chance of overfitting.

Search

- Process of looking for a sequence of actions that reaches the goal.
- search algorithm: takes problem as i/p and outputs action sequence from one node to another node.
 - Agent executes these actions.
- Agent has to drive from point 1 to 2 in a map: Here environment is observable (agent knows current state), discrete (finite action set), known (knows states reached by actions) and deterministic (each action has one outcome).
 - Soln here is a fixed seq. of actions.
 - Agent can ignore percepts in this case.

Data Science and Data Mining 19/2/25

Data science is an interdisciplinary academic field that uses statistics, scientific computing, scientific methods, processing, scientific visualization, algorithms and systems to extract or extrapolate knowledge from potentially noisy, structured, or unstructured data (wiki).



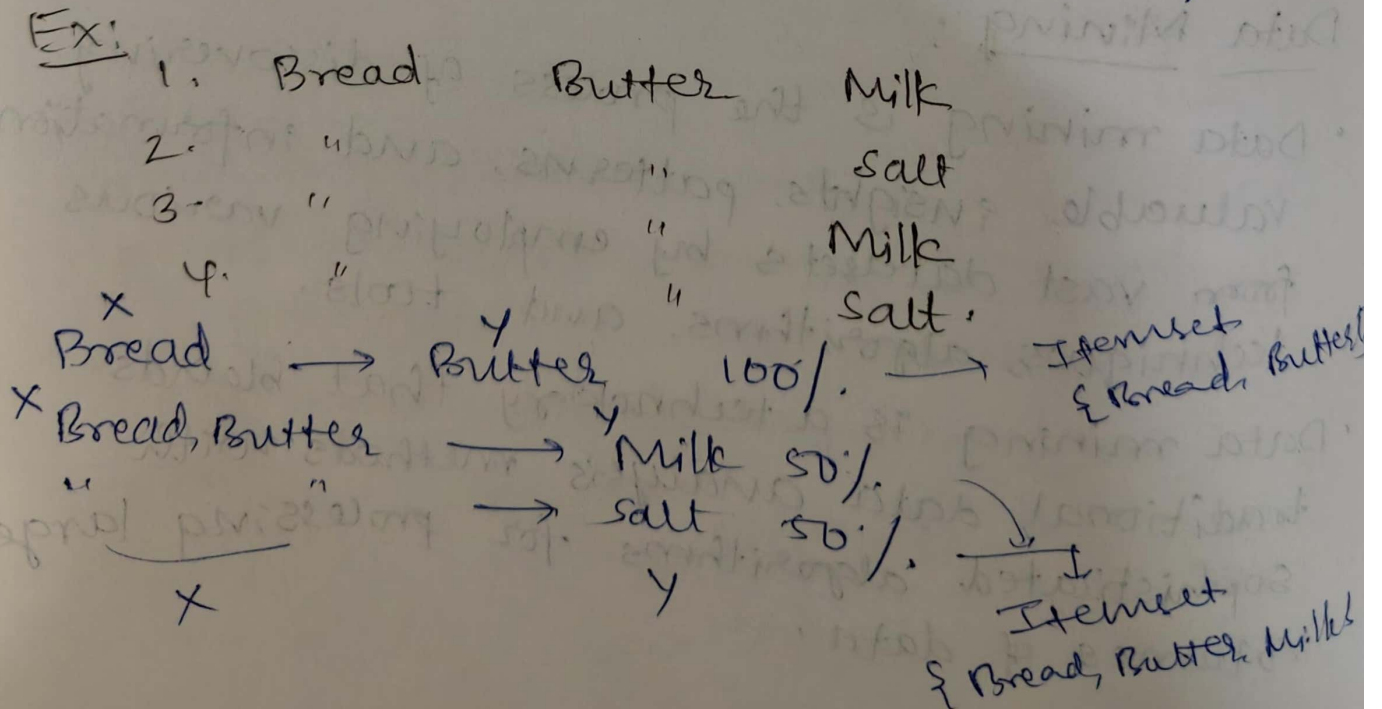
Data Mining :

- Data mining is the process of discovering valuable insights, patterns, and information from vast datasets by employing various techniques, algorithms, and tools.
- Data mining is a technology that blends traditional data analysis methods with sophisticated algorithms for processing large volumes of data.

- In particular, data mining draws upon ideas, such as
 - sampling
 - estimation
 - hypothesis testing from statistics.
 - search algorithms, modeling techniques, and learning theories from artificial intelligence, pattern recognition, and machine learning.

Types of Patterns:

- Associations $X \rightarrow Y$ $\rightarrow$ if we see X , there is high chance of seeing Y .
Ex: Coffee buyers usually also purchase sugar
 - similar are put together
- Clustering
Ex: Segments of customers requiring diff. promotion strategies.
- Classification
Ex: Bank customer is applying for loan, will the applicant be defaulter?



Association Rules:

- antecedent and a consequent
- AR: $X \rightarrow Y$ $\{x_1, x_2, \dots, x_n\} \rightarrow \{y_1, y_2, \dots, y_k\}$
- Y need not be singleton.
- "Itemset" is the list of all the items in the antecedent and the consequent, i.e., $X \cup Y$.
- $\text{supp}(AR) = \text{Prob}(X \cap Y)$ - can't examine this
support - how X, Y are together
Ex: 0.005 - good for large dataset, not for small.
- If AR: $Y \rightarrow X$,
still supp is same.

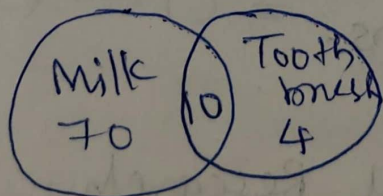
- To evaluate 'association rule' three important measures

$$\text{support}(AR) = \text{Prob}(X \cap Y)$$

$$\frac{\# \text{ transactions containing } X \cap Y}{\# \text{ transactions in } D}$$

$$\text{Confidence}(AR) = \text{Prob}(Y|X)$$

Ex:



X - Tooth brush, Y - Milk

itemset $\{\text{tooth brush, Milk}\}$

$$\text{supp}(AR) = \frac{10}{100} = 0.1$$

$$\text{Confidence}(AR) = \frac{10}{14} = 0.71$$

transactions containing XNY

transactions in X

$$\text{Confidence(AR)} = \text{Prob}(Y|X)$$

$$\text{Lift(AR)} = \frac{\text{Prob}(Y|X)}{\text{Prob}(Y)}$$

transactions containing XNY

transactions in X

transactions in Y

transactions in D

Ex: $\text{Lift(AR)} = \frac{0.7}{0.8} \rightarrow \frac{10/14}{80/100}$

$$\sum_{i=1}^n \binom{n}{i} \times (2^{n-i} - 1)$$

→ no. of associations
→ upper bound
→ lower bound 2^n

Applications of Association Rules: Range $[2^n, 4^n]$

- Market Basket Analysis
 - People who have brought sundara kandam have also brought srinad Bhagaratham.
- Recommendation systems.
- Fraud Detection
- Healthcare and Medical Research
 - Allergy to latex rubber usually co-occurs
- Census analysis with allergies to banana tomato.
- Sports

Transaction Id

1

2

3

4

Tomato
T, P, O - onion

T, P, B, P_u - pumpkin

T, P, O, C - chilly

Ta, L - Lemon

← remainder

AR: Tomato, Potato → Onion

Itemset: {Tomato, Potato, Onion}

$$\text{supp}(AR) = \frac{2}{4} = 1/2 = 0.5$$

$$\text{confidence}(AR) = \frac{2}{3} = 0.66$$

$$\text{Lift} = \frac{2/3}{1/2} = 4/3$$

Challenges in identifying AR's:

• How many association rules are possible with 10 diff items?

• 57002

• This further increases exponentially

• What can we say about

since $\text{Lift}(X \rightarrow Y)$ and $\text{Lift}(Y \rightarrow X)$ by using supp , confidence can't determine exactly?

$$\text{Lift}(X \rightarrow Y) = \frac{\# \text{tr. } X \cap Y}{\# \text{tr. } X}$$

$$\frac{\# \text{tr. } Y}{\# \text{tr. } D}$$

$$= \frac{\# \text{tr. } X \cap Y}{\# \text{tr. } X} \times \frac{\# \text{tr. } D}{\# \text{tr. } Y}$$

$$= \frac{\# \text{tr. } (X \cap Y)}{\# \text{tr. } (Y)} \times \frac{\# \text{tr. } D}{\# \text{tr. } (X)}$$

$$= \text{Lift}(Y \rightarrow X)$$

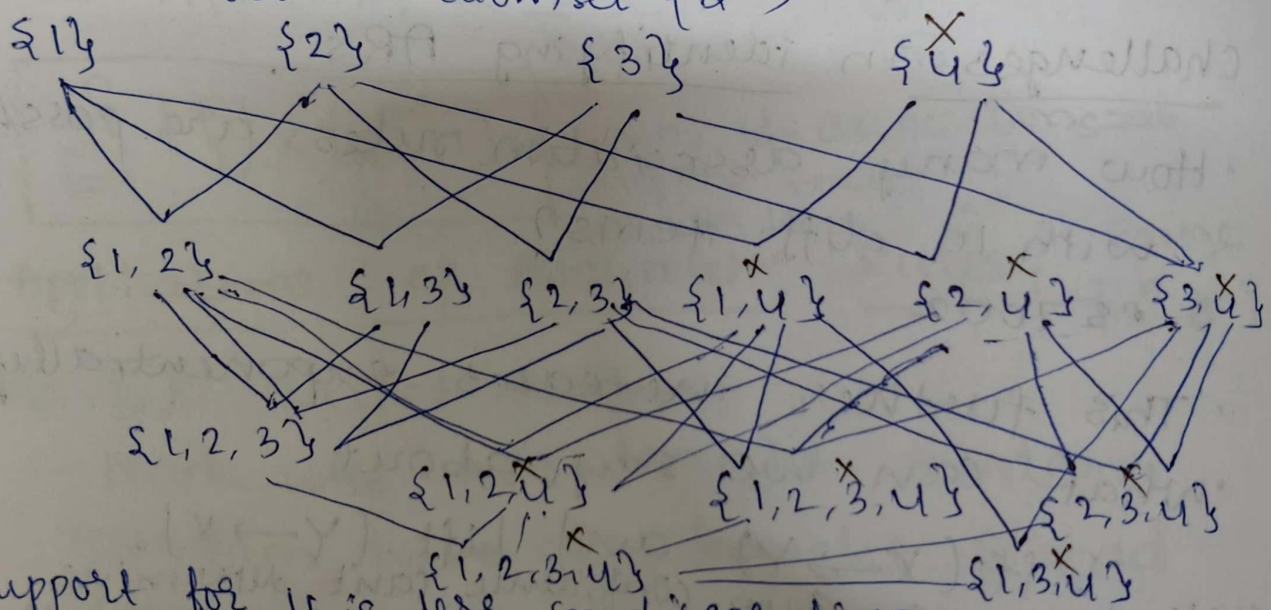
If $\text{supp}(X \rightarrow Y)$ good, $\text{supp}(Y \rightarrow X)$ is

- If $\text{lift} < 1$, may not occur together

- If $\text{lift} > 1$, seeing high chances of

Apriori Algorithm

- Invented by Rakesh Agrawal & Ramakant Srikant (1994).
- Can we speed up than pure brute force?
- Apriori: acknowledges the prior knowledge
 - If any itemset is not frequent, its superset cannot be frequent.
 - An itemset is frequent if its ~~superset~~ ^{subsets} is frequent.
 - An itemset can be frequent only if all its ~~elem~~ subsets are frequent.
 - subset is atomset (2^n)



Support for 4 is less, so discarding.

How does it work?

Step 0: Create 1-size frequent itemsets list that meet threshold support, $k=1$.

Step 1: Expand the itemsets list

- From the k -sized itemsets list combine overlapping sets to $k+1$ size itemsets list.

Step 2: Prune the expanded itemsets list using apriori property.

$k = k+1$

step 3: remove infrequent itemsets from the list.
Repeat step 1, 2, 3 till no more further expansion possible.

- frequent if it appears ^{at least} 3 times in database.

	iter 1	iter 2	
1, 2, 3, 4	{1, 2, 3}	{1, 2, 3}	X
1, 2, 4	{2, 3}	{2, 3}	{1, 2, 4} X
1, 2	{3, 4}	{2, 4}	{2, 3, 4} X
2, 3, 4	{1, 4}	{3, 4}	
2, 3		{1, 3}	} Remove seq. if this exists.
3, 4		{1, 4}	
2, 4			

• $\text{Supp}(X \cup Y) < \text{threshold} \times |D|$ 22/2/25
 $\hookrightarrow$ Discard all the supersets containing this set.

• If we want to consider $(X \cup Y)$, all subsets containing them should also be frequent.

Ex: $k=3$

		①	②	③	
1	1, 2, 3, 4	$\{1\}=3$	$\{1, 2\}=3$	$\{2, 3, 4\}=2$	X - since less than threshold
2	1, 2, 4	$\{2\}=6$	$\{2, 3\}=3$	All subsets should be frequent	
3	1, 2	$\{3\}=4$	$\{2, 4\}=4$		
4	2, 3, 4	$\{4\}=5$	$\{3, 4\}=3$		
5	2, 3		$\{1, 3\}=1$	$\{1, 2, 3\}=1$	X
6	3, 4		$\{1, 4\}=2$	$\{1, 2, 4\}=1$	X
7	2, 4				since $\{1, 4\}$ is discarded $\{4, 3\}$ is not

$\{1, 2\} \& \{3, 4\}$
 - can't combine to get a set of 3 elements. no overlap, size will be 4.

Pseudocode; $\xrightarrow{\text{transactions / database}}$ threshold $\rightarrow$ may be integer/float depending on data

Apriori (T, ϵ) {

$c =$ all '1' in itemsets

while ($|c| > 0$)

{

Find frequency of all itemsets in c

$L =$ sets in c with freq $\geq \epsilon$

$C = \text{AprioriGenerate}(L, k)$

}

}

AprioriGenerate (L, k) $\xrightarrow{\text{starts with 1.}}$

{

$\forall x, y \in L$

if $|x \cup y| = k-1$

x, y

$\Rightarrow$ differ by at most one item.

$z = x \cup y$

if any $k-1$ sized subset of z not in L ,
don't add z to L

else add z to L .

return L

}

Ex:

1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6

1,2

1,3

1,4

1,5

2,3

2,4

2,5

3,4

3,5

4,5

1/2

1/3

1/4

1/5

2/3

2/4

2/5

3/4

3/5

4/5

1,2

1,3

2,4

3,4

3,5

4,5

from step 2
 $(1,2,3) \times \{2,3\}$
 $(1,2,4) \times \{1,4\}$
 $(1,3,4) \times \{1,4\}$
 $(1,3,5) \times \{1,5\}$
 $(2,3,4) \times \{2,3\}$
 $(2,4,5) \times \{2,5\}$
 $3,4,5$

$\Rightarrow \{3,4,5\}$

$L_1: \{1\}, \{2\}, \{3\}, \{4\}, \{5\}$

$L_2: \{1,2\}, \{1,3\}, \{2,4\}, \{3,4\}, \{3,5\}, \{4,5\}$

$L_3: \{3,4,5\}$

WIKIPEDIA'S CODE:

$\text{Apriori}(T, \epsilon) \{$

$L_1 \leftarrow \{ \text{large singleton itemsets} \}$

$k \leftarrow 2$

while L_{k-1} is not empty

$C_k \leftarrow \text{Generate_candidates}(L_{k-1}, k)$
for transactions t in T

$D_t \leftarrow \{c \text{ in } C_k : c \subseteq t\}$

for candidates c in D_t

$\text{count}[c] \leftarrow \text{count}[c] + 1$

$L_k \leftarrow \{c \text{ in } C_k : \text{count}[c] \geq \epsilon\}$

$k \leftarrow k + 1$

return $\text{Union}\{L_k\}$ over all k .

}

$\text{Generate_candidates}(L, k) \{$

$\text{result} \leftarrow \text{empty_set}()$

for all $p \in L, q \in L$ where p and q differ in exactly one element

$c \leftarrow p \cup q$

if $u \in L$ for all $u \subseteq c$ where $|u| = k - 1$

$\text{result.add}(c)$

return result

}

searching
for the
subsets
in the
given
data,

- Different types of association rules (categorical, hierarchical, cyclic)
- Eclat Algorithm.
 - Equivalence class Transformation: a depth-first search strategy.

• FP-growth.

- Frequent Pattern Growth: a compact data structure called the FP-tree (Frequent Pattern tree) to compress the dataset.

→ Data Mining textbook - Available in library

Applications:

• Text classification

- Classify emails into spam/non-spam
- NL

(A page missed). - ~~Mobile~~, ~~Rithvika~~ sent.

• We collect diff. measurements/facts/about certain features.

• $x = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d$, $x = (x^1, \dots, x^n) \in \mathbb{R}^n$

- In the above example, x_1, x_2 are diameter and weight.

• $y \in \{1, 2, \dots, K\} = [K]$ - Label space (Y)

means $(1 \rightarrow K)$ elements set.

$y_i \in Y$

• If $K=2$ binary classification, else, multi-class classification.

• If $K=1$ b/w same class, no information to get from it.

$X \subseteq \mathbb{R}^d$ dataset

$Y = [K]$ label set

- what is classifier?

 $x^{\text{new}} \quad y^{\text{new}}$

Confusion Matrix:

$\hat{y}_i$	0	1
y_i 0	TN	FP
1	FN	TP

Annotations:

- FP (False Positive) is labeled "False positive".
- FN (False Negative) is labeled "false negative".

$$P = FN + TP$$

$$N = TN + FP$$

$$\text{Accuracy} = \frac{TP + TN}{P + N}$$

classified properly

- How to measure 'goodness' of a classifier?
- If only 1% of population has cancer, then a test for cancer that classifies all people as non-cancer will predict 99% of the trials correctly.

$$\text{Recall / Sensitivity} = \frac{TP}{P} = \frac{TP}{TP + FN} \quad (\text{True Positive rate TPR})$$

$$\text{Precision} = \frac{TP}{FP + TP} \quad (\text{Positive prediction value PPV})$$

$$\text{FNR} = \frac{FN}{P} \Rightarrow \text{TPR} + \text{FNR} = 1$$

$$\text{TNR} = \frac{TN}{N} \Rightarrow \text{TNR} + \text{FPR} = 1$$

$$\text{FPR} = \frac{FP}{N}$$

$$\text{TPR} = \frac{TP}{P} \rightarrow \text{Recall / Sensitivity}$$

F_1 - score harmonic mean of recall & precision

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = \frac{2}{\frac{FP + TP}{TP} + \frac{TP + FN}{TP}} = \frac{2TP}{2TP + FP + FN}$$

Applications:

- Text classification:
 - classify emails into spam/non-spam
 - NLP problems
 - Tagging: classify words into verbs, nouns, etc.
- Risk management, fraud detection, Computer intrusion decision.
 - Given the properties of a transaction (items ~~to~~ purchased, amount, location, customer, profile, etc)
 - Determine if it is a fraud.
- Machine learning / pattern recognition applications
 - Vision
 - speech recognition etc.
- All of science & knowledge is about predicting future from past
 - So classification is a very fundamental problem with ultra-wide scope of applications.